# Ensuring Reliability: Error Correction in Semiconductor Devices

Frank, senior SoC design engineer/ ICTK

2026/10/29

# Content


1 **About me** — slides 3–4

Background · fabless chip design · now at ICTK · what an SoC engineer does

2 **Error correction in semiconductor devices** — slides 5–20

- 2.1 **Why ECC?** — noisy channels, errors in real chips, Hamming distance — 6–8
- 2.2 **Parity & check-sum** — parity bit, checksum, CRC with an LFSR — 9–10
- 2.3 **Hamming and SECDED** — fix one error, detect two; PUF keys — 11–15
- 2.4 **Modern codes** — Galois fields, BCH, Reed–Solomon, LDPC, Polar — 16–20

3 **What makes a good employee** — slides 21–23

Six habits companies value · open questions · Q&A

# Part 1 · About me




**Nguyen Manh Duc**

Frank · 원덕만

Ph.D. · Senior SoC Design Engineer, ICTK

**11+**
years in Korea
since 2015

**6+**
years in chip design
fabless, since 2020

**Ph.D.**
University of Ulsan
2020

**3**
languages
VI · EN · KO

## Fabless chip design · 2020–2026

- **Touchscreen controllers:** active-pen algorithms with matched filter and FFT
- **Automotive NPU SoC:** SECDED + CRC engines for functional safety, AXI SRAM controller with ECC
- **Touch & haptic ICs, MCU:** SoC specs with 68 and 136 channels, QSPI chiplet link with CRC
- **Charging PMICs with GaN:** USB Type-C/PD, sigma-delta ADC filters, fuel gauge

## Now · ICTK

security SoCs, since 2026

- BCH ECC for PUF keys (t = 1…9)
- PQC accelerator verification
- Key-manager spec for the next secure element

**One thread:** error correction — my Ph.D. on quantum codes, my chips, and today's lecture

# What an SoC design engineer does: spec to silicon

### 1 Architecture spec

Wrote the spec of a touch SoC with 136 channels and a stylus

*across projects*

### 2 RTL design

Verilog BCH(255,K) codec with t = 1…9 as a parameter

*across projects*

### 3 Verification

Python golden models, NIST test vectors, product firmware run on the RTL

*across projects*

### 4 FPGA prototype

Charging-protocol IP on an FPGA with an analog daughter board

*across projects*

### 5 Synthesis & timing

AXI ECC-SRAM controller at 1,022 MHz in Samsung 5 nm

*across projects*

### 6 Physical design

RTL-to-GDSII (TSMC, DBH, KF, …) for area, timing and power

*across projects*

### 7 Tape-out

Touch IC and MCU; automotive NPU; USB-C PD IC

*across projects*

### 8 Silicon bring-up

Debug silicon with the analog, firmware and test (ATE) teams

*across projects*

**Daily tools:** Verilog / System-Verilog / VHDL · Python · MATLAB · C/Cpp · VCS · Verdi · Verilator · cocotb · SpyGlass · Git · Bash · Claude AI · Xilinx Vivado

PART 2

# Error correction in semiconductor devices

The codes I have put into chips — parity, CRC, Hamming/SECDED, BCH — and the modern ones: Reed–Solomon, LDPC, Polar

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| sent | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| received | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |

**syndrome $0101_2 = 5$**

→ flip bit 5 → data repaired (slide 12)

# Why ECC? Every channel is noisy




## Key ideas

- Code rate **R = k / n ≤ 1**: fraction of bits that carry information.
- **Shannon (1948):** for any rate R below the channel capacity C, a code exists whose error probability → 0 as n → ∞.
- ECC trades redundancy for reliability. No code is 100 % — the goal is an error rate below the system spec.
- In silicon the price is **area, latency and power** of the encoder and decoder.

## 5G NR targets the ECC hardware must meet

| 20 Gb/s | < 1 ms | $< 10^{-5}$ |
|---|---|---|
| eMBB peak throughput | uRLLC latency | uRLLC reliability |

Same story in memory: DRAM, SRAM and flash must return the right data every cycle, at GHz clock rates, for 10+ years.

Source: ISSCC 2018 Tutorial T4, H.-C. Chang, “ECC in 5G/NVM Applications”, slides 4–5, 12; S. Southwell, Introduction to Error Detection and Correction (2022).

# Why ECC? Errors in real chips

RAM

## DRAM & SRAM

- Soft errors: particle strikes flip stored bits
- 64 data + 8 check bits → 72-bit ECC DIMM
- Automotive SRAM (FuSA): SECDED [13,8,4]

LINK

## High-speed links

- LPDDR5 link-ECC:

  Hamming [71,64,3] → SECDED [72,64,4]
- PCIe 6.0 adds FEC to the link
- CRC catches the rest → retry

NVM

## Flash memory

- Cell Vt distributions overlap as cells shrink and wear
- NOR: fast BCH, t = 2–3
- NAND: BCH → LDPC with soft information

5G

## Wireless (5G NR)

- LDPC for data channels (Gb/s)
- Polar codes for control channels
- Turbo codes in 3G / 4G LTE

RS

## Optical & 2-D codes

- CD / DVD and DAT/DDS tape: Reed-Solomon
- QR codes survive dirt and scratches
- Interleaving spreads burst errors

PUF

## Hardware security

- PUF = noisy on-chip fingerprint
- Syndrome stored as helper data in NVM
- ECC regenerates the same key at every power-up

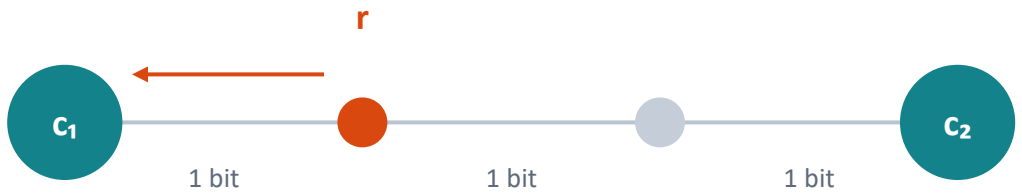
Sources: ISSCC 2018 T4 (slides 11–13, 40–46); Southwell (2022); frank_hamming general.xlsx; vPUF_ECC_hamming_0921.xlsx.

# Core idea: redundancy and Hamming distance

- **(n, k) block code:** k message bits → n-bit codeword. Only $2^k$ of the $2^n$ patterns are valid.
- **Hamming distance d(x, y):** number of bit positions where x and y differ.
- **Minimum distance $d_{min}$:** smallest distance between any two codewords — it sets what the code can do.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| x = | 1 | 0 | 1 | 1 | 1 | 0 | 1 | **d = 2** |
| y = | 1 | 0 | 0 | 1 | 0 | 0 | 1 | |

**d_min = 3 → one error stays closest to the original**




received r is decoded to the nearest codeword $c_1$

**Detect up to d_min − 1 errors · correct up to ⌊(d_min − 1)/2⌋**

| d_min | Detect | Correct | Example |
|---|---|---|---|
| 2 | 1 | 0 | Parity |
| 3 | 2 | 1 | Hamming |
| 4 | 3 | 1 (+ detect 2) | SECDED |
| 2t + 1 | 2t | t | BCH, RS |

### Detect → retransmit (ARQ)

Parity · checksum · CRC — Ethernet, USB, PCIe replay

### Forward error correction (FEC)

Hamming · BCH · RS · LDPC · Polar — memory, flash, 5G, storage: no time (or no way) to ask again

Source: Southwell (2022) Part 1 “Hamming codes”; ecc_vn_bch_full.pdf §4.1; ISSCC 2018 T4 slide 6.

# Parity: the simplest error detector

## Even parity for one byte

| D0 | D1 | D2 | D3 | D4 | D5 | D6 | D7 | P |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |

Five 1s in the data → P = 1, so the 9-bit word has an even number of 1s.

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Sent | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | check = 0 ✓ OK |
| 1 error | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | check = 1 → detected |
| 2 errors | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | check = 0 ✗ missed! |

**Receiver:** XOR all 9 bits. 0 → accept, 1 → error detected (ask again).

**Blind spot:** any even number of flips looks valid.

```
// Parity for one byte (Verilog)
wire [7:0] din;
wire p_even =  ^din;   // XOR-reduce
wire p_odd  = ~^din;   // XNOR-reduce
// receiver: err = ^{din_rx, p_rx};
```

## Properties

- Detects any odd number of bit errors; cannot locate → cannot correct
- Cost: 1 bit per word — 12.5 % for 8 b, 6.25 % for 16 b (but longer words raise the odds of 2 errors)
- Hardware: XOR tree — 7 XOR2 gates, 3 levels for 8 bits
- Fits rare, random errors: UART, cache/TLB tags, register files

Source: Southwell (2022) Part 1 “Parity”; error_detection_correction.pptx slide 3; ecc_vn_bch_full.pdf §2.1.

# Check-sum and CRC: stronger detection

## Checksum

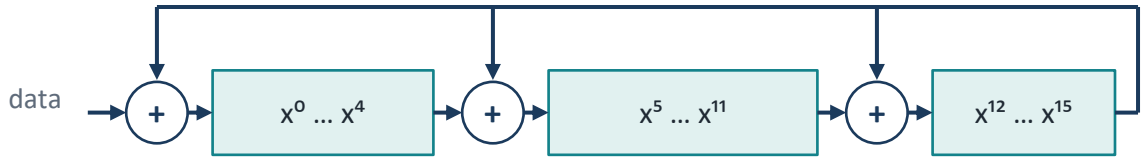
- Add all data words modulo $2^n$ (n = checksum width)
- Append the two's complement → receiver's sum over block + checksum = 0
- 8-bit checksum: 1 in 256 corrupted blocks still pass
- **No position information** — reordering words gives the same sum

```
data     : 0x12 0x34 0x56
sum      : 0x9C      chk = -sum = 0x64
receiver : 0x9C + 0x64 = 0x100 → 0x00 ✓

swapped  : 0x56 0x34 0x12
sum      : 0x9C  → passes ✗ (undetected)
```

**Real use:** IPv4 / TCP / UDP headers carry a 16-bit one's-complement checksum — cheap in software, backed by the Ethernet CRC-32 underneath.

## CRC — Cyclic Redundancy Check



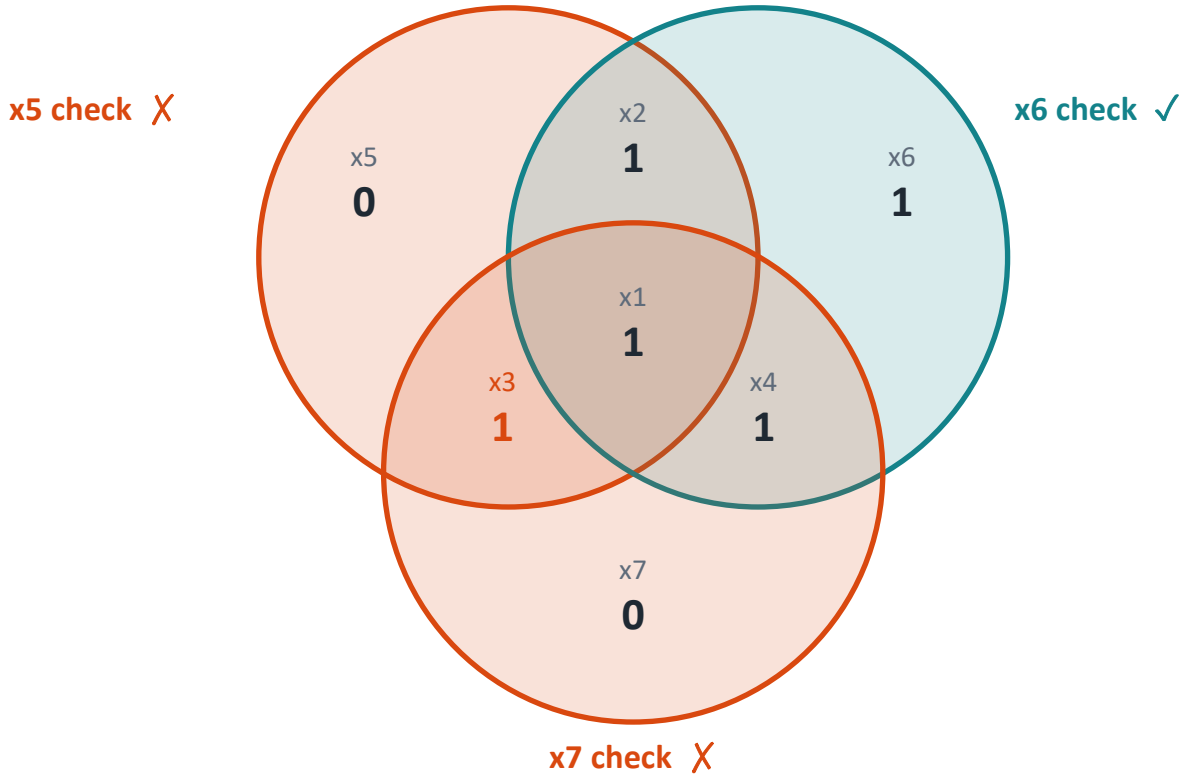

**CRC-CCITT LFSR: $g(x) = x^{16} + x^{12} + x^5 + 1$**

- Shift the message through the LFSR → the register holds the remainder of $M(x)\cdot x^r \div g(x)$ (mod-2 division)
- Receiver divides again: remainder ≠ 0 → error. Catches every burst ≤ r bits and is order-sensitive
- Wide buses: parallel XOR matrix / LUT computes one word per clock

| Standard | g(x) | Used in |
|---|---|---|
| CRC-16-CCITT | $x^{16}+x^{12}+x^5+1$ | X.25, HDLC, Bluetooth |
| CRC-16 | $x^{16}+x^{15}+x^2+1$ | USB, Modbus |
| CRC-32 | 0x04C11DB7 (refl. 0xEDB88320) | Ethernet, ZIP, PNG |

Source: Southwell (2022) Part 1 “Checksum”, “CRC”; error_detection_correction.pptx slide 4; ecc_vn_bch_full.pdf §3.

# Hamming (7,4,3): parity circles point to the error




Received r = 1111010: circles x5 and x7 fail, x6 passes → only x3 is inside both failing circles and outside the passing one → flip x3.

## Encoder: three parity equations

```
x5 = x1 ⊕ x2 ⊕ x3
x6 = x1 ⊕ x2 ⊕ x4
x7 = x1 ⊕ x3 ⊕ x4
```

1. **Encode:** u = (1101) → v = u·G = (1101 010)
2. **Channel:** flips x3 → r = (1111 010)
3. **Check:** re-compute the 3 parities → pattern ✗ ✓ ✗ = syndrome
4. **Correct:** the syndrome names x3 → flip it → v recovered

| 4/7 | 3 | 1 bit |
|---|---|---|
| code rate | d_min | corrected |

Source: ISSCC 2018 T4 slide 6 ((7,4) Hamming example, v = 1101010, r = 1111010); R. W. Hamming, Bell Syst. Tech. J. (1950).

# Hamming (12,8,3): syndrome = error position

| Position | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Bit** | P1 | P2 | D0 | P4 | D1 | D2 | D3 | P8 | D4 | D5 | D6 | D7 |
| **P1 covers** | ● | | ● | | ● | | ● | | ● | | ● | |
| **P2 covers** | | ● | ● | | | ● | ● | | | ● | ● | |
| **P4 covers** | | | | ● | ● | ● | ● | | | | | ● |
| **P8 covers** | | | | | | | | ● | ● | ● | ● | ● |
| **Codeword** | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| **Received** | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |

## Decoder: re-compute the four parities on r

```
s0 = ⊕ pos{1,3,5,7,9,11}   = 1
s1 = ⊕ pos{2,3,6,7,10,11}  = 0
s2 = ⊕ pos{4,5,6,7,12}     = 1
s3 = ⊕ pos{8,9,10,11,12}   = 0
```

**s3 s2 s1 s0 = $0101_2$ = 5**

→ flip bit 5 (D1) and the word is repaired

- Parity bits sit at positions 1, 2, 4, 8; $P_i$ covers every position whose index has bit i set.
- Check bits r: $2^r \geq k + r + 1$ → k = 8 needs r = 4.

Source: frank_hamming general.xlsx (sheet “Hamming 12,8”); vPUF_ECC_hamming_0921.xlsx; Southwell (2022) Part 1. Data D0..D7 = 0100 1111.

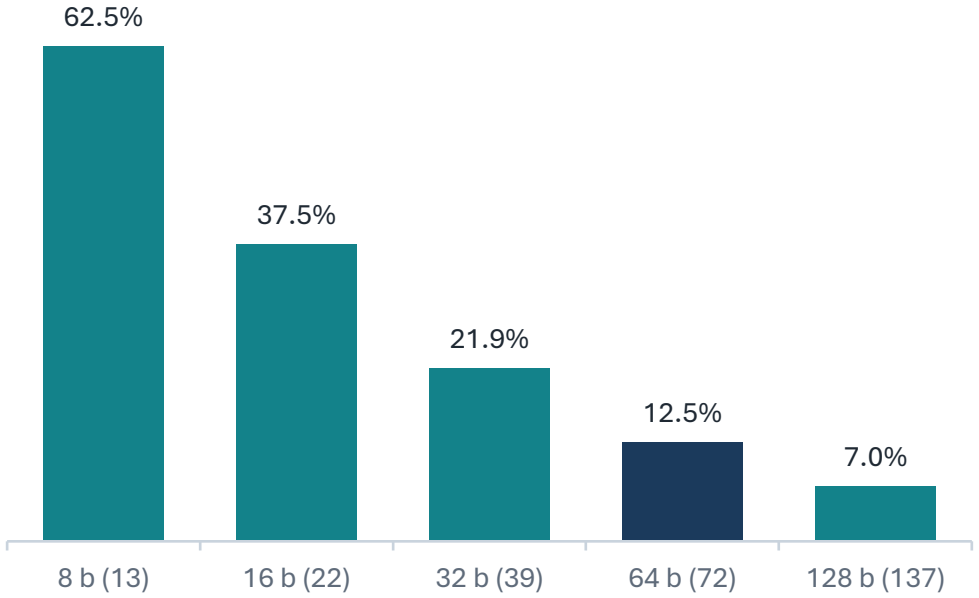
# SECDED: correct one error, detect two

Overall parity bit P0 over all 12 bits → d_min = 4 → **SECDED [13, 8, 4]**

| Syndrome s | Overall parity | Diagnosis | Action |
|---|---|---|---|
| 0 | OK | no error | pass data |
| ≠ 0 | fail | single error at s | flip bit s → corrected |
| 0 | fail | error in P0 only | data OK, log it |
| ≠ 0 (or > 12) | OK | double error | flag uncorrectable |

- Errors at 5 and 7 → s = 2 (a wrong bit!) but overall parity is OK → flagged instead of mis-corrected
- Hamming (d = 3) is a cyclic code; adding P0 breaks cyclicity but keeps linearity
- Hsiao code (odd-weight columns): same power, fewer and balanced XORs → common in DRAM controllers

**SECDED check-bit overhead vs data width**




**64 + 8 = 72 bits** → why ECC DIMMs and RLDRAM parts come in ×9 / ×18 / ×36 widths.

**Shipping examples:** SRAM FuSA [13,8,4] · LPDDR5 link-ECC [71,64,3] → [72,64,4]

Source: Southwell (2022) Part 1; frank_hamming general.xlsx (sheets “Hamming 12,8”, “Hamming 71,64”); ecc_vn_bch_full.pdf §4.3–4.5.

# ECC at work: PUF — a chip's fingerprint

## What is a PUF?

- **Physically Unclonable Function:** a digital fingerprint from uncontrollable manufacturing variation (Vth, delay, resistance, via formation)
- Challenge C → response R = $f_{chip}(C)$: same design, same C, different chip → different R
- The key is not stored — it is re-derived from the silicon when needed

**Entropy sources**

Arbiter | Ring osc. | SRAM | Latch / BF | NAND | **Via**

### Security requirements (ISO/IEC 20897)

| Randomness | Uniqueness | **Steadiness** | Unclonability | Tamper resistance |
|---|---|---|---|---|
| responses are unpredictable | two chips never agree | same chip → same answer | cannot be modelled or copied | probing / SCA breaks it |

## Applications

**ID — Identification**

Public device ID: trace every chip, stop over-production and counterfeits

**C/R — Challenge–response authentication**

Verifier stores CRPs at enrollment, later accepts R' if it is close to R (threshold → FAR / FRR)

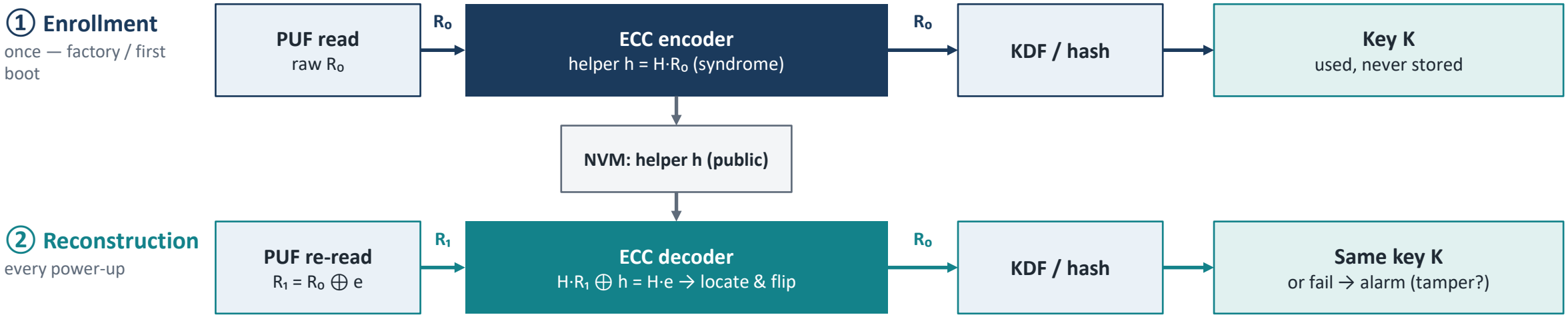
**KEY — Key generation → Root of Trust**

Crypto keys, nonces, RNG seeds for secure boot, device identity, IoT & automotive

**The catch — steadiness.** ≈ 5 % of cells are unstable from small mismatch, ≈ 10 % across −40 °C … 125 °C. Authentication can tolerate a few flipped bits — a cryptographic key tolerates none. → Stabilise the raw bits, then let ECC fix the rest (next slide).

Sources: D. Jeon, "Physical Unclonable Function (PUF)" Part I (slides 5–14, 16–36, 38–46) & Part II (slides 5–6), 2022; ICTK Blog (en.ictk.com/Blog), 2026.

# PUF key generation: enrollment and reconstruction




### Case study: Hamming (12,8) syndrome as helper data

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $R_0$ enroll | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | $s_0 = 5$ |
| $R_1$ run | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | $s_1 = 9$ |

**Enroll:** $s_0$ = H·$R_0$ = 0101 → stored in NVM as helper data

**Run-time:** $s_1$ = H·$R_1$ = 1001

$s_0 \oplus s_1$ = 1100 = **12** → flip bit 12 → $R_0$ restored → same key

### Design notes

- **Stabilise first:** majority voting (TMV/SMV), dark-bit masking, burn-in — ECC then fixes the rest
- **Helper data is public** but leaks r bits per block → use extra PUF bits / an extractor
- **Beyond t errors** reconstruction fails — can flag tampering
- **VIA PUF™ (ICTK):** via connected / not connected at fab → stable bits, key rebuilt without ECC or helper data

Sources: D. Jeon, PUF Part II (slides 7, 13–18, 33), 2022; ICTK Blog “Why Does a PUF Give a Different Reading Every Time?” (2026); vPUF_ECC_hamming_0921.xlsx.

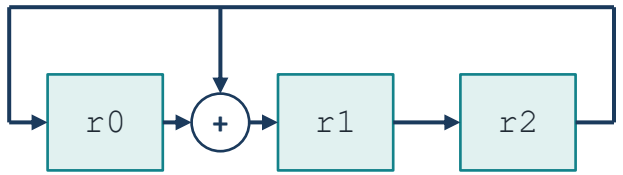
# Math toolbox: Galois fields and cyclic codes

## $GF(2^3)$, $p(x) = x^3 + x + 1$

| Power | Polynomial | Binary |
|---|---|---|
| $\alpha^1$ | $\alpha$ | 010 |
| $\alpha^2$ | $\alpha^2$ | 100 |
| $\alpha^3$ | $\alpha + 1$ | 011 |
| $\alpha^4$ | $\alpha^2 + \alpha$ | 110 |
| $\alpha^5$ | $\alpha^2 + \alpha + 1$ | 111 |
| $\alpha^6$ | $\alpha^2 + 1$ | 101 |
| $\alpha^7$ | 1 | 001 |

- **Add** = bitwise XOR: 011 ⊕ 110 = 101
- **Multiply** = add exponents mod 7: $\alpha^2 \cdot \alpha^3 = \alpha^5$ → 100 × 011 = 111
- **Every non-zero element has an inverse** → we can solve for error locations and values

## Hardware: ×α is an LFSR




Start at 001 and clock:

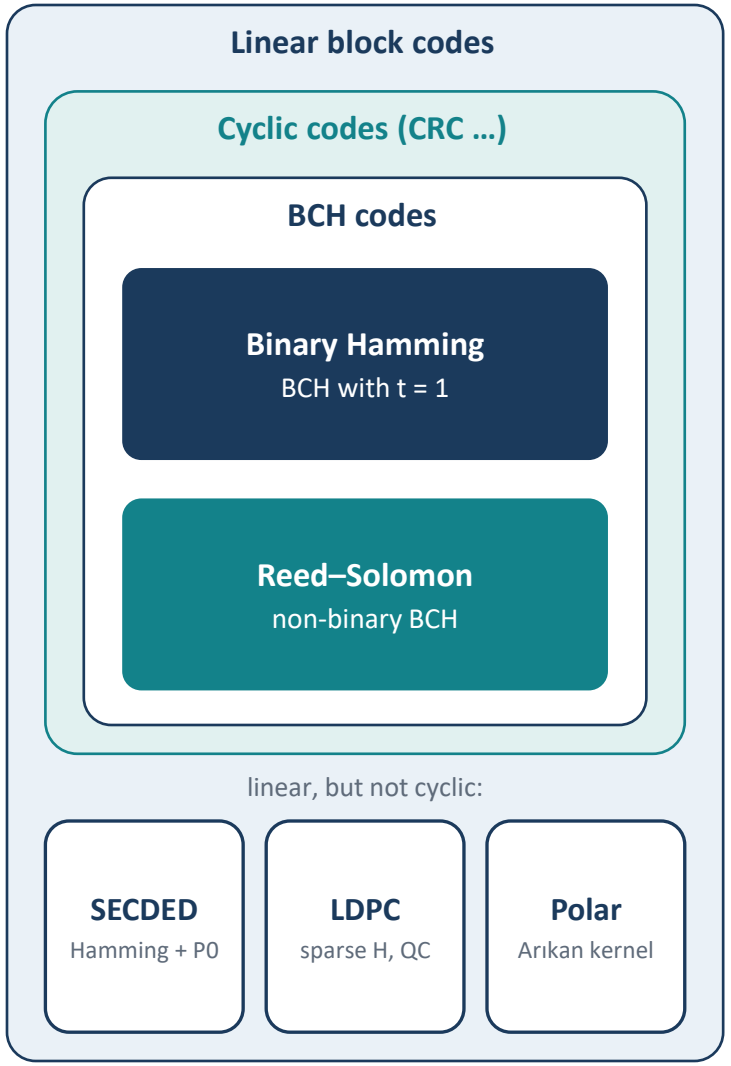
001→010→100→011→110→111→101→001

It walks through all 7 non-zero field elements — the same circuit family as CRC.

### Cyclic code

Any cyclic shift of a codeword is again a codeword.

Codewords = multiples of a generator g(x) that divides $x^n - 1$ → encoder is an LFSR dividing by g(x).




Source: ecc_vn_bch_full.pdf ch. 5–6; Southwell (2022) Part 2 “Galois field”; error_detection_correction.pptx slide 13.

# BCH codes: correcting t random bit errors

- **Bose, Ray-Chaudhuri, Hocquenghem** (1959–60): cyclic codes for t errors
- Roots $\alpha, \alpha^2, \ldots, \alpha^{2t}$ in g(x) ⇒ **$d_{min} \geq 2t + 1$**
- **$g(x) = LCM\{ m_1(x), m_3(x), \ldots, m_{2t-1}(x) \}$** (minimal polynomials)

**Example: BCH(15,7), t = 2, GF($2^4$), p(x) = $x^4+x+1$**

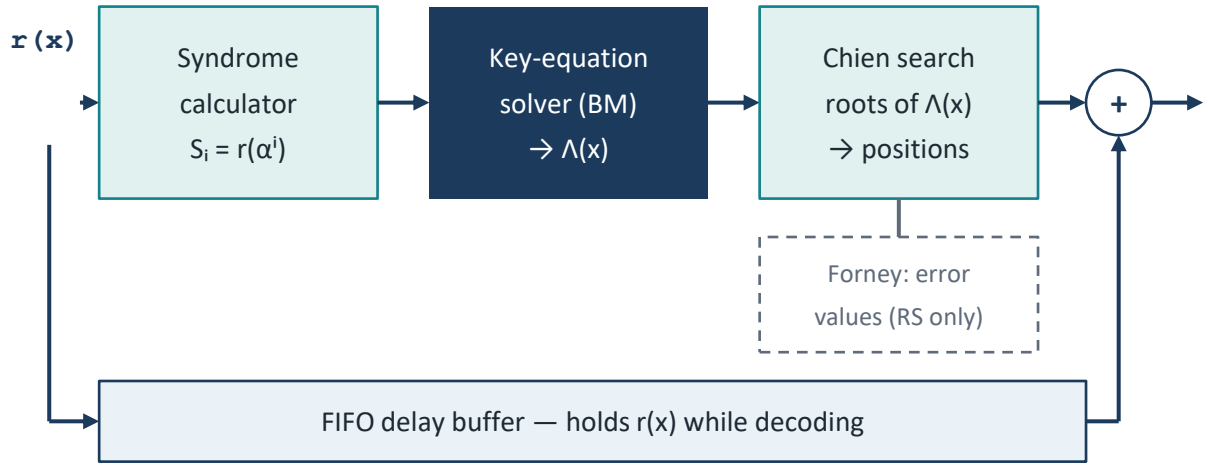
$$m_1(x) = x^4 + x + 1$$
$$m_3(x) = x^4 + x^3 + x^2 + x + 1$$
$$g(x) = m_1 \cdot m_3 = x^8+x^7+x^6+x^4+1$$

**deg g = 8 → 8 parity bits, k = 15 − 8 = 7**

| n = 15 | t = 1 | t = 2 | t = 3 |
|---|---|---|---|
| code | (15,11) | (15,7) | (15,5) |
| rate | 0.73 | 0.47 | 0.33 |

**Encoder = LFSR dividing by g(x):** 8 flip-flops + 4 XORs

## Decoder pipeline (BCH & RS)




### NOR flash

Fully-parallel BCH(274,256), t = 2

**12.5 K gates · 2.5 ns**

90 nm codec

### NAND SSD

Multi-channel BCH decoder

**124-bit correctability**

Binary BCH: position known → just flip the bit

Source: ISSCC 2018 T4 slides 41–43, 50; ecc_vn_bch_full.pdf ch. 6 (BCH(15,7) worked example, encoder/decoder RTL).

# Reed–Solomon: correcting whole symbols (bursts)

- Works on m-bit symbols in GF($2^m$); length $n \le 2^m - 1$
- **2t parity symbols → t symbol errors** (or 2t erasures)
- A burst of up to m bits inside one symbol counts as one error
- RS(255,223) over GF($2^8$): 32 parity bytes → 16-byte errors
- Decoder = BCH pipeline + Forney (error value)

## Worked example: RS(7,5) over GF($2^3$) — 5 data + 2 parity symbols

| A | B | C | D | E | P | Q |
|---|---|---|---|---|---|---|
| 101 | 100 | 010 | 100 | 111 | 100 | 100 |
| 101 | 100 | 110 | 100 | 111 | 100 | 100 |

sent (row 1) → received (row 2): symbol C corrupted — 1, 2 or 3 flipped bits make no difference

$S0 = A \oplus B \oplus C' \oplus D \oplus E \oplus P \oplus Q = 100$ (= error pattern)
$S1 = \alpha^7 A \oplus \alpha^6 B \oplus \alpha^5 C' \oplus \ldots \oplus \alpha^1 Q = 001$
$\alpha^k = S1 / S0 = \alpha^7 / \alpha^2 = \alpha^5 \rightarrow k = 5 \rightarrow$ symbol C
$C = C' \oplus S0 = 110 \oplus 100 = 010$ ✓

S0 tells **what** went wrong, S1/S0 tells **where** — division = subtracting logs

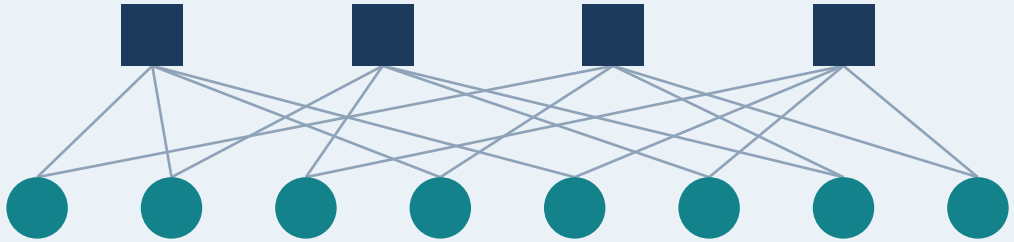
### Where you meet RS

- QR codes · CD / DVD
- DAT/DDS tape: product code RS(32,28) × RS(32,26)
- Outer code of storage & broadcast systems

Source: Southwell (2022) Part 2 & error_detection_correction.pptx slides 10–15 (RS(7,5) after J. Watkinson, The Art of Digital Audio); ecc_vn_bch_full.pdf ch. 7–8.

# Capacity-approaching codes: LDPC and Polar

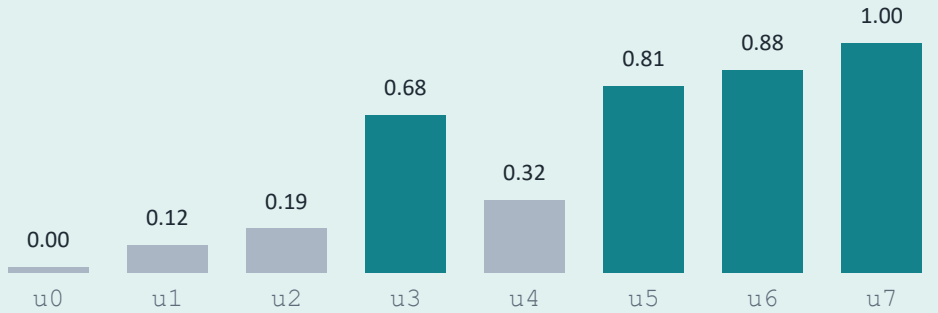
## LDPC — Low-Density Parity-Check



■ check nodes = rows of H ● bit nodes = codeword bits

- Gallager 1960, rediscovered by MacKay (1990s); sparse H, Tanner graph
- Belief propagation: check ↔ bit messages, iterate until all checks pass
- Uses soft information → near Shannon limit; 5G NR data, Wi-Fi, NAND SSD
- **Silicon: (2048,1920) decoder, 11.5 Gb/s, 191 mW, 90 nm**

## Polar codes




■ information ■ frozen (known 0) N = 8, BEC(0.5)

- Arıkan 2009: first provably capacity-achieving code, low-complexity enc/dec
- Bit-channels polarize to ~perfect or ~useless → send data on good ones only
- SC / SC-List + CRC decoding; 5G NR control, strong for short blocks (< 1000 b)

### Hard vs soft decoding on a NAND page: BCH (t = 150) vs LDPC — probability the page decodes

| Error bits | 110 | 130 | 150 | 170 | 190 |
|---|---|---|---|---|---|
| **BCH (t = 150)** | 100 % OK | 100 % OK | 100 % OK | 100 % fail | 100 % fail |
| **LDPC** | 99.9999 % | 99.999 % | 99.99 % | 95 % | 90 % |

BCH: deterministic cliff at t.

LDPC: graceful, probabilistic — never 100 %, so real systems add CRC/retry.

Source: ISSCC 2018 T4 slides 15–16 (LDPC), 21–26 (Polar), 47 (BCH vs LDPC), 53 (CP-PEG LDPC chip). Polar bars computed for BEC(0.5).

# Summary: choosing an ECC as an IC designer

| Code | Corrects / detects | Typical use | Hardware cost |
|---|---|---|---|
| **Parity** | detects odd # of errors | UART, cache tags | XOR tree |
| **Checksum / CRC** | detects; CRC: all bursts ≤ r | Ethernet, USB, PCIe | adder / LFSR |
| **Hamming / SECDED** | 1 bit fix, 2 bit detect | SRAM, DRAM, links | XOR trees, 1 cycle |
| **BCH** | t random bits | NOR / NAND flash, PUF | syndrome + BM + Chien |
| **Reed–Solomon** | t symbols (bursts) | QR, CD/DVD, tape | GF($2^8$) + Forney |
| **LDPC** | near capacity, soft | 5G data, SSD, Wi-Fi | iterative BP, big memory |
| **Polar** | capacity-achieving | 5G control | SC / SCL decoder |

## Take-aways

1. **Know your channel**

   random bits, bursts, or soft reliability?

2. **Everything is a trade-off**

   redundancy ↔ strength ↔ area, latency, power

3. **ECC is never the only guard**

   beyond t, decoders mis-correct silently — keep a CRC on top

**Further reading:** H.-C. Chang, ISSCC 2018 Tutorial T4 · S. Southwell, Intro to Error Detection & Correction (2022) & slides · ecc_vn_bch_full.pdf (VN, with BCH chapter) · frank_hamming general.xlsx · vPUF_ECC_hamming_0921.xlsx · D. Jeon, PUF Part I/II (2022) · ICTK Blog

PART 3

# What makes a good employee

Six habits I learned in Korean fabless chip companies — each with a proof from my own work

1. Fundamentals
2. Ownership
3. Verify
4. Communicate
5. Teamwork
6. Keep learning

# What makes a good employee: six habits

## 1 Strong fundamentals

Tools change every few years; math, logic and signals do not.

**In my work:** the Galois-field algebra of Part 2 is the BCH codec I designed at ICTK.

## 2 Ownership

Own the result, not the task: done = designed, verified, documented, integrated.

**In my work:** carried my blocks through tape-out on touch, NPU and charging chips.

## 3 Verify, then trust

Prove it with tests and a reference model. A bug found after tape-out costs months and a new mask set.

**In my work:** running the product firmware on the RTL exposed a race in one RTL condition.

## 4 Clear communication

Write specs others can build from. Report problems early, with data.

**In my work:** architecture specs, test guides, 67 test-vector reports.

## 5 Teamwork across disciplines

Chips are built by analog, digital, firmware, layout and test engineers together.

**In my work:** analog front-end control designed with the analog team.

## 6 Keep learning

Products change; the ability to learn fast is what lasts.

**In my work:** touch → automotive NPU → charging PMIC → security and PQC in six years.

# Thank you · 감사합니다

Questions and discussion

**OPEN QUESTIONS**

1

Your phone has DRAM, flash, a 5G modem and a USB-C port.
→ Which code protects each one?

2

Why must a PUF key be corrected to zero errors?
When a fingerprint login can accept a few?

3

What would you like to build in your first job?

Frank (Nguyen Manh Duc) · Senior SoC Design Engineer, ICTK